

Letao SSO登录功能白盒测试教程

目录

- [1. 测试基础概念](#)
- [2. 测试环境准备](#)
- [3. 白盒测试设计方法](#)
- [4. 实战演练：登录功能测试](#)
- [5. 测试代码实现](#)
- [6. 测试执行与验证](#)

1. 测试基础概念

1.1 什么是白盒测试？

白盒测试（又称结构测试、透明盒测试）是一种基于代码内部结构的测试方法。

核心特点：

- ☒ 测试人员需要了解代码的实现细节
- ☒ 关注代码的逻辑结构（if语句、循环、异常处理等）
- ☒ 分析代码的执行路径
- ☒ 考虑变量的取值范围
- ☒ 理解方法之间的调用关系

对比理解：

- 白盒测试：**像医生做体检,需要了解人体内部结构
- 黑盒测试：**像病人看病,只关注症状和结果,不关心内部

1.2 为什么需要白盒测试？

好处：

- 提高代码质量：**发现逻辑错误、边界问题
- 保证代码覆盖率：**确保每行代码都被测试到
- 降低维护成本：**早期发现问题,修复成本更低
- 增强信心：**有测试保护的代码更安全

适用场景：

- 核心业务逻辑（如登录、支付、订单处理）
- 包含复杂判断的代码
- 对可靠性要求高的功能

1.3 白盒测试的覆盖标准

白盒测试有多种覆盖标准,从简单到复杂依次为：

覆盖标准	目标	测试用例数量	覆盖程度	推荐场景
语句覆盖	每个可执行语句至少执行一次	最少	★	快速验证
判定覆盖	每个if语句的真假分支都执行	较少	★★	一般项目
条件覆盖	每个条件的真假值都执行	较多	★★★	复杂逻辑
判定-条件覆盖	同时满足判定覆盖和条件覆盖	较多	★★★★	重要功能
基本路径覆盖	每条独立执行路径都执行	最多	★★★★★	核心业务

选择建议：

- 🔗 初学者：从语句覆盖开始,逐步过渡到基本路径覆盖
- 🔗 实际项目：推荐使用基本路径覆盖,确保代码质量
- 🔗 核心业务逻辑：使用判定-条件覆盖或基本路径覆盖

1.4 测试对象的选择

问题：白盒测试用例设计是只针对某个类吗？

答案：不是的。白盒测试用例设计可以针对任何代码,但需要根据测试目标选择合适的测试对象。

选择原则：

- ☒ 优先选择包含复杂逻辑的代码（如多个条件判断、循环、异常处理）
- ☒ 优先选择核心业务逻辑（如登录、支付、订单处理）
- ☒ 优先选择对外暴露的接口（如Controller、Service）

本项目选择：

- 重点测试：UserServiceImpl.login 方法（核心业务逻辑,包含多个条件判断）
- 简要分析：AuthController.login 方法（API接口层,包含参数验证、异常处理）

2. 测试环境准备

2.1 项目结构

```
letao-springboot3/
├── letao-sso/
│   ├── src/
│   │   ├── main/java/com/letao/sso/
│   │   │   ├── controller/AuthController.java
│   │   │   ├── service/impl/UserServiceImpl.java
│   │   │   ├── mapper/UserMapper.java
│   │   │   └── utils/JwtUtils.java
│   │   └── test/
│   │       ├── java/com/letao/sso/
│   │       └── controller/AuthControllerTest.java
```

```
|      | └─ service/UserServiceImplTest.java
|      └─ resources/
└─ pom.xml
```

2.2 测试依赖配置

在 `pom.xml` 中添加以下测试依赖：

```
<!-- JUnit 5 - 测试框架 -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.8.2</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <version>5.8.2</version>
  <scope>test</scope>
</dependency>

<!-- Mockito - Mock框架 -->
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>4.5.1</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <version>4.5.1</version>
  <scope>test</scope>
</dependency>

<!-- Spring Boot Test - Spring测试支持 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

依赖说明:

依赖	作用	为什么需要
JUnit 5	提供测试框架和断言	编写和执行测试用例
Mockito	提供Mock功能	模拟依赖对象,隔离测试
Spring Boot Test	集成Spring测试支持	在Spring环境中测试

配置Maven Surefire插件:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.0.0-M7</version>
</plugin>
```

2.3 测试依赖知识讲解

2.3.1 JUnit 5 是什么?

JUnit 5 是Java最流行的单元测试框架。

核心概念:

- **测试方法**: 使用 `@Test` 注解标记的方法
- **断言**: 验证测试结果是否符合预期
- **生命周期**: 测试方法的执行顺序

常用注解:

<code>@Test</code>	// 标记测试方法
<code>@BeforeEach</code>	// 每个测试方法前执行
<code>@AfterEach</code>	// 每个测试方法后执行
<code>@BeforeAll</code>	// 所有测试方法前执行 (只执行一次)
<code>@AfterAll</code>	// 所有测试方法后执行 (只执行一次)

常用断言:

<code>assertEquals(expected, actual)</code>	// 验证相等
<code>assertNull(object)</code>	// 验证为null
<code>assertNotNull(object)</code>	// 验证不为null
<code>assertTrue(condition)</code>	// 验证为true
<code>assertFalse(condition)</code>	// 验证为false

2.3.2 Mockito 是什么?

Mockito 是一个强大的Mock框架,用于模拟依赖对象。

为什么需要Mock?

- **隔离测试**: 避免依赖外部资源 (数据库、网络等)
- **控制行为**: 可以控制依赖方法的返回值
- **提高速度**: 不需要真正访问数据库或网络

核心概念:

- **Mock对象**: 模拟的真实对象
- **Stubbing**: 设置Mock对象的行为
- **Verification**: 验证Mock对象是否被调用

常用方法:

```
// 创建Mock对象
UserMapper userMapper = Mockito.mock(UserMapper.class);

// 设置Mock行为
Mockito.when(userMapper.findByUsername("admin"))
    .thenReturn(user);

// 验证方法调用
Mockito.verify(userMapper).findByUsername("admin");
```

2.3.3 Spring Boot Test 是什么？

Spring Boot Test 提供了在Spring环境中进行测试的支持。

核心注解：

```
@SpringBootTest          // 启动完整的Spring上下文
@ExtendWith(MockitoExtension.class) // 启用Mockito支持
@MockBean                // 替换Spring容器中的Bean为Mock对象
```

使用场景：

- 需要测试Spring Bean的集成
- 需要使用Spring的依赖注入
- 需要测试Controller层

3. 白盒测试设计方法

3.1 白盒测试设计六步法

白盒测试用例设计可以按照以下6个步骤进行：

```
步骤1：阅读和理解代码
↓
步骤2：识别代码结构特征
↓
步骤3：绘制控制流图（可选）
↓
步骤4：识别测试点
↓
步骤5：设计测试用例
↓
步骤6：优化和合并测试用例
```

3.2 步骤1：阅读和理解代码

3.2.1 被测方法代码

```
@Override
public User login(String username, String password) {
    // 步骤1：根据用户名查询用户
    User user = userMapper.findByUsername(username);

    // 步骤2：判断用户是否存在
```

```
    if (user == null) {
        return null;
    }

    // 步骤3: 验证密码是否正确
    if (!passwordEncoder.matches(password, user.getPassword())) {
        return null;
    }

    // 步骤4: 验证用户状态
    if (user.getStatus() != null && user.getStatus() == 0) {
        return null;
    }

    // 步骤5: 返回用户对象
    return user;
}
```

3.2.2 理解代码功能

功能： 用户登录验证

输入：

- `username`：用户名 (String类型)
- `password`：密码 (String类型)

输出：

- 成功：返回User对象
- 失败：返回null

依赖：

- `userMapper`：数据库查询接口
- `passwordEncoder`：密码加密验证接口

执行流程：

1. 根据用户名查询用户信息
2. 如果用户不存在,返回null
3. 如果密码不匹配,返回null
4. 如果用户状态为禁用 (status=0) ,返回null
5. 如果所有验证都通过,返回用户对象

3.3 步骤2：识别代码结构特征

3.3.1 结构特征识别清单

结构特征	识别方法	UserServiceImpl.login方法
条件判断 (if语句)	查找if、else if、三元运算符	3个if语句
循环结构	查找for、while、do-while	无
异常处理	查找try-catch、throw	无
方法调用	查找方法调用语句	2个依赖方法调用
返回语句	查找return语句	4个return语句
复合条件	查找&&、 、!	1个复合条件

3.3.2 UserServiceImpl.login方法的结构特征

条件判断：

- 第1个if: `if (user == null)` - 判断用户是否存在
- 第2个if: `if (!passwordEncoder.matches(...))` - 判断密码是否正确
- 第3个if: `if (user.getStatus() != null && user.getStatus() == 0)` - 判断用户状态

方法调用：

- `userMapper.findByUsername(username)` - 查询用户
- `passwordEncoder.matches(password, user.getPassword())` - 验证密码

返回语句：

- `return null;` - 用户不存在时返回
- `return null;` - 密码错误时返回
- `return null;` - 用户禁用时返回
- `return user;` - 验证通过时返回

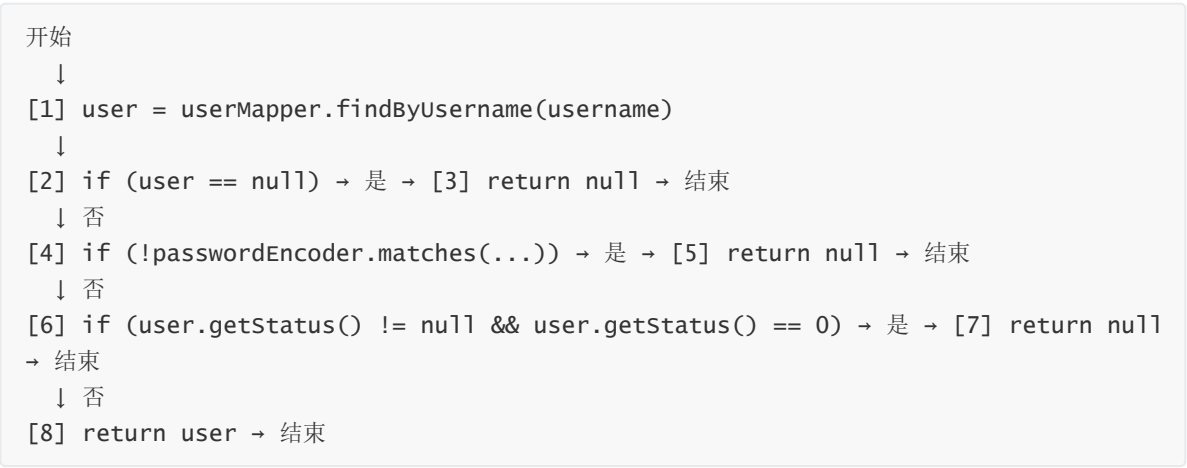
复合条件：

- `user.getStatus() != null && user.getStatus() == 0` - 用户状态判断

3.4 步骤3：绘制控制流图（可选）

3.4.1 控制流图

控制流图用于可视化代码的执行路径,帮助我们理解代码的执行流程。



3.4.2 执行路径识别

根据控制流图,我们可以识别出以下执行路径:

路径编号	执行路径	描述
路径1	1→2(否)→4(否)→6(否)→8	正常登录成功
路径2	1→2(是)→3	用户不存在
路径3	1→2(否)→4(是)→5	密码错误
路径4	1→2(否)→4(否)→6(是)→7	用户禁用

说明:

- 路径1: 所有验证都通过,返回User对象
- 路径2: 用户不存在,返回null
- 路径3: 密码错误,返回null
- 路径4: 用户状态为禁用,返回null

3.5 步骤4: 识别测试点

3.5.1 测试点识别方法

测试点是指代码中需要测试的关键位置或条件。我们可以使用以下方法识别测试点:

测试点类型	识别方法	示例
条件真值	每个if条件为真时	user == null
条件假值	每个if条件为假时	user != null
边界值	数值边界、空值、null	status = 0, status = null
复合条件	&&、 的各个子条件	status != null, status == 0
方法调用	依赖方法的返回值	userMapper.findByUsername()返回null或User对象
异常情况	可能抛出异常的代码	无

3.5.2 测试点识别推导过程

让我们详细展示如何根据"测试点识别方法"一步步推导出测试点。

原始代码:

```
@Override
public User login(String username, String password) {
    // 第1个if条件
    User user = userMapper.findByUsername(username);
    if (user == null) return null; // ← 条件1

    // 第2个if条件
    if (!passwordEncoder.matches(password, user.getPassword())) return null; // ← 条件2

    // 第3个if条件 (复合条件)
```



```
    if (user.getStatus() != null && user.getStatus() == 0) return null; // ← 条件3

    return user;
}
```

推导1: 应用"方法调用"识别方法

识别方法: 查找方法调用语句,考虑依赖方法的返回值

代码中的方法调用:

- `userMapper.findByUsername(username)` - 第1行
- `passwordEncoder.matches(password, user.getPassword())` - 第4行

推导测试点:

- **测试点1.1:** `userMapper.findByUsername()` 返回什么值?
 - 可能返回null (用户不存在)
 - 可能返回User对象 (用户存在)
- **测试点1.2:** `passwordEncoder.matches()` 返回什么值?
 - 可能返回true (密码匹配)
 - 可能返回false (密码不匹配)

结论: 得到2个测试点

- 测试点1: 用户查询结果 (null或User对象)
 - 测试点2: 密码验证结果 (true或false)
-

推导2: 应用"条件真值"识别方法

识别方法: 每个if条件为真时的情况

代码中的if条件:

- 条件1: `user == null`
- 条件2: `!passwordEncoder.matches(password, user.getPassword())`
- 条件3: `user.getStatus() != null && user.getStatus() == 0`

推导测试点:

- **测试点2.1:** 条件1为真时,即 `user == null`
 - 需要 `userMapper.findByUsername()` 返回null
- **测试点2.2:** 条件2为真时,即 `!passwordEncoder.matches(...)` 为真
 - 需要 `passwordEncoder.matches()` 返回false
- **测试点2.3:** 条件3为真时,即 `user.getStatus() != null && user.getStatus() == 0` 为真
 - 需要 `user.getStatus() != null` 为真且 `user.getStatus() == 0` 为真
 - 即status不为null且status等于0

结论: 得到3个测试点 (与测试点1、测试点2合并)

- 测试点1: 用户查询结果为null
 - 测试点2: 密码验证结果为false
 - 测试点3: 用户状态为0
-

推导3：应用"条件假值"识别方法

识别方法：每个if条件为假时的情况

代码中的if条件：

- 条件1: `user == null`
- 条件2: `!passwordEncoder.matches(password, user.getPassword())`
- 条件3: `user.getStatus() != null && user.getStatus() == 0`

推导测试点：

- **测试点3.1**：条件1为假时,即 `user != null`
 - 需要 `userMapper.findByUsername()` 返回User对象
- **测试点3.2**：条件2为假时,即 `!passwordEncoder.matches(...)` 为假
 - 需要 `passwordEncoder.matches()` 返回true
- **测试点3.3**：条件3为假时,即 `user.getStatus() != null && user.getStatus() == 0` 为假
 - 情况A: `user.getStatus() != null` 为假 (即status为null)
 - 情况B: `user.getStatus() != null` 为真但 `user.getStatus() == 0` 为假 (即status不为null且status不等于0)

结论：得到3个测试点 (与测试点1、测试点2、测试点3合并)

- 测试点1：用户查询结果为User对象
 - 测试点2：密码验证结果为true
 - 测试点3：用户状态为null或不为0
-

推导4：应用"边界值"识别方法

识别方法：查找数值边界、空值、null

代码中的变量和常量：

- `user.getStatus()` - 可能的值: null、0、1、2、...
- `username` - 可能的值: null、空字符串、有效用户名
- `password` - 可能的值: null、空字符串、有效密码

推导测试点：

- **测试点4.1**：status的边界值
 - `status = null` (空值)
 - `status = 0` (禁用状态,边界值)
 - `status = 1` (正常状态)
 - `status = 2` (其他状态)
- **测试点4.2**：username的边界值
 - `username = null`
 - `username = ""` (空字符串)
 - `username = "admin"` (有效用户名)
- **测试点4.3**：password的边界值
 - `password = null`
 - `password = ""` (空字符串)
 - `password = "123456"` (有效密码)

结论：得到3个测试点

- 测试点3: 用户状态值 (null、0、1、2)
- 测试点4: 用户名边界值 (null、空字符串、有效值)
- 测试点5: 密码边界值 (null、空字符串、有效值)

注意: 测试点4和测试点5在UserServiceImpl.login方法中不需要测试,因为:

- username和password的null检查在AuthController层完成
 - UserServiceImpl.login方法假设username和password已经通过验证
-

推导5: 应用"复合条件"识别方法

识别方法: 查找&&、||、!,分析各个子条件

代码中的复合条件:

- 条件3: `user.getStatus() != null && user.getStatus() == 0`

子条件分析:

- 子条件3.1: `user.getStatus() != null`
- 子条件3.2: `user.getStatus() == 0`

推导测试点:

- **测试点5.1:** 子条件3.1为真
 - `status != null`
- **测试点5.2:** 子条件3.1为假
 - `status == null`
- **测试点5.3:** 子条件3.2为真
 - `status == 0`
- **测试点5.4:** 子条件3.2为假
 - `status != 0`

组合测试点 (考虑&&的短路特性):

- **组合1:** 子条件3.1为真,子条件3.2为真
 - `status != null && status == 0`
 - 即`status = 0`
- **组合2:** 子条件3.1为真,子条件3.2为假
 - `status != null && status != 0`
 - 即`status = 1`或`status = 2`等
- **组合3:** 子条件3.1为假 (子条件3.2不执行,因为&&短路)
 - `status == null`

结论: 得到3个测试点 (与测试点3合并)

- 测试点3: `status = 0` (组合1)
 - 测试点3: `status = 1`或`2` (组合2)
 - 测试点3: `status = null` (组合3)
-

推导6: 应用"异常情况"识别方法

识别方法: 查找可能抛出异常的代码

代码分析:

- `userMapper.findByUsername(username)` - 可能抛出数据库异常
- `passwordEncoder.matches(password, user.getPassword())` - 可能抛出加密异常

推导测试点：

- **测试点6.1**：数据库连接异常
- **测试点6.2**：密码加密异常

结论：得到2个测试点

- 测试点6：数据库异常
- 测试点7：密码加密异常

注意：这些异常测试点在UserServiceImpl.login方法中不需要测试,因为：

- 异常处理在AuthController层完成
- UserServiceImpl.login方法假设依赖方法不会抛出异常

3.5.3 最终测试点汇总

经过以上6个推导步骤,我们得到以下测试点：

测试点	推导来源	具体内容
测试点1 ：用户查询结果	推导1（方法调用） 推导2（条件真值） 推导3（条件假值）	- userMapper.findByUsername()返回null - userMapper.findByUsername()返回User对象
测试点2 ：密码验证结果	推导1（方法调用） 推导2（条件真值） 推导3（条件假值）	- passwordEncoder.matches()返回true - passwordEncoder.matches()返回false
测试点3 ：用户状态值	推导2（条件真值） 推导3（条件假值） 推导4（边界值） 推导5（复合条件）	- status = null - status = 0 - status = 1 - status = 2（其他状态）
测试点4 ：用户名边界值	推导4（边界值）	在UserServiceImpl中不需要测试
测试点5 ：密码边界值	推导4（边界值）	在UserServiceImpl中不需要测试
测试点6 ：数据库异常	推导6（异常情况）	在UserServiceImpl中不需要测试
测试点7 ：密码加密异常	推导6（异常情况）	在UserServiceImpl中不需要测试

最终保留的测试点：

测试点1：用户查询结果

- `userMapper.findByUsername()`返回null（用户不存在）
- `userMapper.findByUsername()`返回User对象（用户存在）

测试点2：密码验证结果

- `passwordEncoder.matches()`返回true（密码正确）
- `passwordEncoder.matches()`返回false（密码错误）

测试点3：用户状态值

- `status = null`（状态未设置）
- `status = 0`（用户禁用）
- `status = 1`（用户正常）
- `status = 2`（其他状态值）

测试点4：复合条件的子条件

- `user.getStatus() != null`为真,`user.getStatus() == 0`为假
- `user.getStatus() != null`为真,`user.getStatus() == 0`为真
- `user.getStatus() != null`为假

3.6 步骤5：设计测试用例

3.6.1 测试用例设计的基本规则

规则1：测试用例必须包含的要素

每个测试用例必须包含以下要素：

要素	说明	示例
测试用例ID	唯一标识符	TC_ST_001
测试场景	测试的目的和场景	正常登录成功
输入参数	方法的输入值	<code>username="admin", password="123456"</code>
预期输出	方法的预期返回值	返回User对象
前置条件	测试执行前需要满足的条件	数据库中存在admin用户
Mock行为	模拟依赖方法的返回值	<code>userMapper.findByUsername()</code> 返回testUser
覆盖的测试点	该测试用例覆盖的测试点	测试点1、测试点2、测试点3

规则2：测试用例设计原则

原则	说明	示例
独立性	每个测试用例应该独立运行,不依赖其他测试用例	每个测试用例都独立设置Mock行为
可重复性	测试用例应该可以重复执行,结果一致	使用固定的测试数据
明确性	测试用例的输入和预期输出必须明确	清晰指定username和password的值
完整性	测试用例应该覆盖所有测试点	确保每个测试点至少被一个测试用例覆盖
最小化	测试用例数量应该尽可能少,但覆盖完整	合并重复的测试用例

规则3：测试用例设计流程

1. 确定测试目标（覆盖标准）

↓

2. 选择测试点

↓

3. 设计输入参数

↓

4. 确定预期输出

↓

5. 设计Mock行为

↓

6. 验证测试用例

3.6.2 不同覆盖标准的测试用例设计

3.6.2.1 语句覆盖测试用例

目标：找到一条能执行所有语句的路径

分析：

- 要执行所有语句,需要走完整个方法
- 路径：userMapper.findByUsername() → if (user == null) → if (!passwordEncoder.matches()) → if (user.getStatus() != null && user.getStatus() == 0) → return user
- 需要：user != null, passwordEncoder.matches()返回true, status条件为假

设计测试用例：

要素	值
测试用例ID	TC_ST_001
测试场景	正常登录成功
输入参数	username="admin", password="123456"
预期输出	返回User对象
Mock行为	userMapper.findByUsername()返回User对象(status=1) passwordEncoder.matches()返回true
覆盖的测试点	测试点1（用户存在）、测试点2（密码正确）、测试点3（status=1）
触发路径	路径1：1→2(否)→4(否)→6(否)→8

3.6.2.2 判定覆盖测试用例

目标：每个if语句的真假分支都执行

分析：

- 有3个if语句,需要覆盖每个if的真假分支
- if1: user == null（需要真和假）
- if2: !passwordEncoder.matches()（需要真和假）
- if3: user.getStatus() != null && user.getStatus() == 0（需要真和假）

设计测试用例：

测试用例1：覆盖if1的真分支

要素	值
测试用例ID	TC_DEC_001
测试场景	用户不存在
输入参数	username="notexist", password="任意"
预期输出	返回null
Mock行为	userMapper.findByUsername()返回null
覆盖的测试点	测试点1（用户不存在）
触发路径	路径2：1→2(是)→3
覆盖的判定	if1(真)

测试用例2：覆盖if1的假分支、if2的真分支

要素	值
测试用例ID	TC_DEC_002
测试场景	密码错误
输入参数	username="admin", password="wrong"
预期输出	返回null
Mock行为	userMapper.findByUsername()返回User对象 passwordEncoder.matches()返回false
覆盖的测试点	测试点1（用户存在）、测试点2（密码错误）
触发路径	路径3：1→2(否)→4(是)→5
覆盖的判定	if1(假)、if2(真)

测试用例3：覆盖if1的假分支、if2的假分支、if3的真分支

要素	值
测试用例ID	TC_DEC_003
测试场景	用户禁用
输入参数	username="admin", password="123456"
预期输出	返回null
Mock行为	userMapper.findByUsername()返回User对象(status=0) passwordEncoder.matches()返回true
覆盖的测试点	测试点1（用户存在）、测试点2（密码正确）、测试点3（status=0）
触发路径	路径4：1→2(否)→4(否)→6(是)→7
覆盖的判定	if1(假)、if2(假)、if3(真)

测试用例4：覆盖if1的假分支、if2的假分支、if3的假分支

要素	值
测试用例ID	TC_DEC_004
测试场景	正常登录
输入参数	username="admin", password="123456"
预期输出	返回User对象
Mock行为	userMapper.findByUsername()返回User对象(status=1) passwordEncoder.matches()返回true
覆盖的测试点	测试点1（用户存在）、测试点2（密码正确）、测试点3（status=1）
触发路径	路径1：1→2(否)→4(否)→6(否)→8
覆盖的判定	if1(假)、if2(假)、if3(假)

判定覆盖总结：

测试用例	if1	if2	if3
TC_DEC_001	真	-	-
TC_DEC_002	假	真	-
TC_DEC_003	假	假	真
TC_DEC_004	假	假	假

☒ 所有if语句的真假分支都被覆盖

3.6.2.3 条件覆盖测试用例

目标：每个条件的真假值都执行

分析：

- 复合条件：user.getStatus() != null && user.getStatus() == 0
- 子条件1：user.getStatus() != null
- 子条件2：user.getStatus() == 0
- 需要覆盖每个子条件的真值和假值

设计测试用例：

测试用例1：子条件1为真,子条件2为真

要素	值
测试用例ID	TC_CON_001
测试场景	用户禁用
输入参数	username="admin", password="123456"
预期输出	返回null
Mock行为	userMapper.findByUsername()返回User对象(status=0) passwordEncoder.matches()返回true
覆盖的测试点	测试点3 (status=0) 、测试点4 (子条件1真、子条件2真)
条件覆盖	user.getStatus() != null(真)、 user.getStatus() == 0(真)

测试用例2：子条件1为真,子条件2为假

要素	值
测试用例ID	TC_CON_002
测试场景	用户正常
输入参数	username="admin", password="123456"
预期输出	返回User对象
Mock行为	userMapper.findByUsername()返回User对象(status=1) passwordEncoder.matches()返回true
覆盖的测试点	测试点3 (status=1) 、测试点4 (子条件1真、子条件2假)
条件覆盖	user.getStatus() != null(真)、 user.getStatus() == 0(假)

测试用例3：子条件1为假（子条件2不执行,因为&&短路）

要素	值
测试用例ID	TC_CON_003
测试场景	用户状态为null
输入参数	username="admin", password="123456"
预期输出	返回User对象
Mock行为	userMapper.findByUsername()返回User对象(status=null) passwordEncoder.matches()返回true
覆盖的测试点	测试点3 (status=null) 、测试点4 (子条件1假)
条件覆盖	user.getStatus() != null(假)、 user.getStatus() == 0(不执行)

条件覆盖总结：

测试用例	user.getStatus() != null	user.getStatus() == 0
TC_CON_001	真	真
TC_CON_002	真	假
TC_CON_003	假	不执行

☒ 所有子条件的真值都被覆盖（假值通过短路特性覆盖）

3.6.2.4 基本路径覆盖测试用例

目标： 每条独立执行路径都执行

分析：

- 有4条独立执行路径
- 路径1：1→2(否)→4(否)→6(否)→8（正常登录）
- 路径2：1→2(是)→3（用户不存在）
- 路径3：1→2(否)→4(是)→5（密码错误）
- 路径4：1→2(否)→4(否)→6(是)→7（用户禁用）

设计测试用例：

测试用例1：覆盖路径1

要素	值
测试用例ID	TC_PATH_001
测试场景	正常登录
输入参数	username="admin", password="123456"
预期输出	返回User对象
Mock行为	userMapper.findByUsername()返回User对象(status=1) passwordEncoder.matches()返回true
覆盖的测试点	测试点1（用户存在）、测试点2（密码正确）、测试点3（status=1）
触发路径	路径1：1→2(否)→4(否)→6(否)→8

测试用例2：覆盖路径2

要素	值
测试用例ID	TC_PATH_002
测试场景	用户不存在
输入参数	username="notexist", password="任意"
预期输出	返回null
Mock行为	userMapper.findByUsername()返回null
覆盖的测试点	测试点1（用户不存在）
触发路径	路径2：1→2(是)→3

测试用例3：覆盖路径3

要素	值
测试用例ID	TC_PATH_003
测试场景	密码错误
输入参数	username="admin", password="wrong"
预期输出	返回null
Mock行为	userMapper.findByUsername()返回User对象 passwordEncoder.matches()返回false
覆盖的测试点	测试点1（用户存在）、测试点2（密码错误）
触发路径	路径3：1→2(否)→4(是)→5

测试用例4：覆盖路径4

要素	值
测试用例ID	TC_PATH_004
测试场景	用户禁用
输入参数	username="admin", password="123456"
预期输出	返回null
Mock行为	userMapper.findByUsername()返回User对象(status=0) passwordEncoder.matches()返回true
覆盖的测试点	测试点1（用户存在）、测试点2（密码正确）、测试点3（status=0）
触发路径	路径4：1→2(否)→4(否)→6(是)→7

基本路径覆盖总结：

测试用例	触发路径	覆盖的测试点
TC_PATH_001	路径1	测试点1、测试点2、测试点3
TC_PATH_002	路径2	测试点1
TC_PATH_003	路径3	测试点1、测试点2
TC_PATH_004	路径4	测试点1、测试点2、测试点3

☒ 所有独立执行路径都被覆盖

3.7 步骤6：优化和合并测试用例

3.7.1 优化示例

原始测试用例	优化后测试用例	合并原因
TC1: user == null	TC1: 用户不存在	合并多个条件为真的情况
TC2: user != null	TC2: 密码错误	合并多个条件为假的情况
TC3: passwordEncoder.matches()返回false	TC3: 用户禁用	合并status条件测试
TC4: passwordEncoder.matches()返回true	TC4: 正常登录	合并正常情况测试
TC5: status条件为真	-	已合并到TC3
TC6: status条件为假	-	已合并到TC4

3.7.2 最终测试用例汇总

基本路径覆盖的测试用例（推荐使用）：

测试用例ID	测试场景	输入参数	预期输出	Mock行为	覆盖的路径
TC_PATH_001	正常登录	username="admin", password="123456"	返回User对象	userMapper返回User(status=1) passwordEncoder返回true	路径1
TC_PATH_002	用户不存在	username="notexist", password="任意"	返回null	userMapper返回null	路径2
TC_PATH_003	密码错误	username="admin", password="wrong"	返回null	userMapper返回User passwordEncoder返回false	路径3
TC_PATH_004	用户禁用	username="admin", password="123456"	返回null	userMapper返回User(status=0) passwordEncoder返回true	路径4

4. 实战演练：登录功能测试

4.1 完整实例演示

让我们通过一个完整的实例来演示从代码到测试用例的完整过程。

4.1.1 原始代码

```
public User login(String username, String password) {  
    User user = userMapper.findByUsername(username);  
    if (user == null) return null;  
    if (!passwordEncoder.matches(password, user.getPassword())) return null;  
    if (user.getStatus() != null && user.getStatus() == 0) return null;  
    return user;  
}
```

4.1.2 思考过程

思考1：这个方法有哪些可能的执行路径？

- 路径A：用户不存在 → 返回null
- 路径B：用户存在,密码错误 → 返回null
- 路径C：用户存在,密码正确,状态禁用 → 返回null
- 路径D：用户存在,密码正确,状态正常 → 返回User对象

思考2：如何触发路径A？

- 需要userMapper.findByUsername()返回null
- 测试用例：username="不存在的用户"

思考3：如何触发路径B？

- 需要userMapper.findByUsername()返回User对象
- 需要passwordEncoder.matches()返回false
- 测试用例：username="admin", password="错误密码"

思考4：如何触发路径C？

- 需要userMapper.findByUsername()返回User对象 (status=0)
- 需要passwordEncoder.matches()返回true
- 测试用例：username="admin", password="正确密码", status=0

思考5：如何触发路径D？

- 需要userMapper.findByUsername()返回User对象 (status=1)
- 需要passwordEncoder.matches()返回true
- 测试用例：username="admin", password="正确密码", status=1

思考6：是否还有其他需要测试的情况？

- status为null的情况 (复合条件的子条件测试)
- 测试用例：username="admin", password="正确密码", status=null

4.1.3 最终测试用例

测试用例	输入	预期输出	触发路径
TC1	username="不存在", password="任意"	null	路径A
TC2	username="admin", password="错误"	null	路径B
TC3	username="admin", password="正确", status=0	null	路径C
TC4	username="admin", password="正确", status=1	User对象	路径D
TC5	username="admin", password="正确", status=null	User对象	路径D（子条件测试）

4.2 白盒测试用例设计总结

4.2.1 核心思路

1. **理解代码**：先理解代码的功能和逻辑
2. **识别结构**：找出条件判断、循环、异常等结构
3. **绘制路径**：绘制控制流程图,识别执行路径
4. **识别测试点**：找出需要测试的关键点（条件真值、边界值等）
5. **设计用例**：为每个测试点设计具体的测试用例
6. **优化合并**：合并重复用例,提高测试效率

4.2.2 关键技巧

- **从简单到复杂**：先设计语句覆盖,再设计路径覆盖
- **从整体到细节**：先识别主要路径,再关注边界条件
- **从正向到反向**：先测试正常情况,再测试异常情况
- **从独立到组合**：先测试单个条件,再测试条件组合

4.2.3 常见误区

- ✗ 只测试正常情况,忽略异常情况
- ✗ 只测试条件真值,忽略条件假值
- ✗ 只测试单个条件,忽略条件组合
- ✗ 只测试方法返回值,忽略方法调用
- ☑ 全面覆盖所有执行路径和条件组合

通过以上思考过程,我们可以系统地设计出完整的白盒测试用例,确保代码的正确性和健壮性。

5. 测试代码实现

5.1 UserServiceImpl测试代码实现

```
package com.letao.sso.service.impl;

import com.letao.entity.User;
import com.letao.sso.mapper.UserMapper;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
```

```

import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.crypto.password.PasswordEncoder;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.*;

/**
 * UserServiceImpl单元测试
 * 测试目标：基本路径覆盖
 */
@ExtendWith(MockitoExtension.class)
class UserServiceImplTest {

    @Mock
    private UserMapper userMapper;

    @Mock
    private PasswordEncoder passwordEncoder;

    @InjectMocks
    private UserServiceImpl userService;

    private User testUser;

    @BeforeEach
    void setUp() {
        // 初始化测试用户对象
        testUser = new User();
        testUser.setId(1L);
        testUser.setUsername("admin");

        testUser.setPassword("$2a$10$N.zmdr9k7uOCQb376NoUnuTJ8iAt6Z5EHsM8lE9lBosl7iAt6Z5EH");
        testUser.setStatus(1);
    }

    /**
     * 测试用例1：正常登录
     * 覆盖路径：路径1（用户存在 + 密码正确 + status正常）
     */
    @Test
    void testLogin_Success() {
        // Given
        String username = "admin";
        String password = "123456";

        when(userMapper.findByUsername(username)).thenReturn(testUser);
        when(passwordEncoder.matches(password,
testUser.getPassword())).thenReturn(true);

        // When
        User result = userService.login(username, password);

        // Then

```



```

        assertNotNull(result);
        assertEquals("admin", result.getUsername());
        assertEquals(1, result.getStatus());

        verify(userMapper).findByUsername(username);
        verify(passwordEncoder).matches(password, testUser.getPassword());
    }

    /**
     * 测试用例2: 用户不存在
     * 覆盖路径: 路径2 (用户不存在)
     */
    @Test
    void testLogin_UserNotFound() {
        // Given
        String username = "notexist";
        String password = "any";

        when(userMapper.findByUsername(username)).thenReturn(null);

        // when
        User result = userService.login(username, password);

        // Then
        assertNull(result);

        verify(userMapper).findByUsername(username);
        verify(passwordEncoder, never()).matches(anyString(), anyString());
    }

    /**
     * 测试用例3: 密码错误
     * 覆盖路径: 路径3 (用户存在 + 密码错误)
     */
    @Test
    void testLogin_WrongPassword() {
        // Given
        String username = "admin";
        String password = "wrong";

        when(userMapper.findByUsername(username)).thenReturn(testUser);
        when(passwordEncoder.matches(password,
testUser.getPassword())).thenReturn(false);

        // when
        User result = userService.login(username, password);

        // Then
        assertNull(result);

        verify(userMapper).findByUsername(username);
        verify(passwordEncoder).matches(password, testUser.getPassword());
    }

    /**
     * 测试用例4: 用户禁用
     * 覆盖路径: 路径4 (用户存在 + 密码正确 + status禁用)
     */

```

```

@Test
void testLogin_UserDisabled() {
    // Given
    String username = "admin";
    String password = "123456";
    testUser.setStatus(0);

    when(userMapper.findByUsername(username)).thenReturn(testUser);
    when(passwordEncoder.matches(password,
testUser.getPassword())).thenReturn(true);

    // when
    User result = userService.login(username, password);

    // Then
    assertNull(result);

    verify(userMapper).findByUsername(username);
    verify(passwordEncoder).matches(password, testUser.getPassword());
}

/**
 * 测试用例5：用户状态为null
 * 覆盖路径：路径5（用户存在 + 密码正确 + status=null）
 */
@Test
void testLogin_StatusNull() {
    // Given
    String username = "admin";
    String password = "123456";
    testUser.setStatus(null);

    when(userMapper.findByUsername(username)).thenReturn(testUser);
    when(passwordEncoder.matches(password,
testUser.getPassword())).thenReturn(true);

    // when
    User result = userService.login(username, password);

    // Then
    assertNotNull(result);
    assertEquals("admin", result.getUsername());
    assertNull(result.getStatus());

    verify(userMapper).findByUsername(username);
    verify(passwordEncoder).matches(password, testUser.getPassword());
}
}

```

5.2 测试代码讲解

5.2.1 注解说明

```
@ExtendWith(MockitoExtension.class) // 启用Mockito支持
```

- 作用：启用Mockito的Mock功能
- 为什么需要：使用Mockito需要先启用这个扩展

```
@Mock  
private UserMapper userMapper;
```

- 作用：创建UserMapper的Mock对象
- 为什么需要：模拟数据库操作,避免真正访问数据库

```
@InjectMocks  
private UserServiceImpl userService;
```

- 作用：创建UserServiceImpl对象,并自动注入@Mock标记的依赖
- 为什么需要：自动将Mock对象注入到被测对象中

```
@BeforeEach  
void setup() {  
    // 初始化测试数据  
}
```

- 作用：在每个测试方法执行前执行
- 为什么需要：为每个测试方法准备相同的测试数据

```
@Test  
void testLogin_Success() {  
    // 测试方法  
}
```

- 作用：标记测试方法
- 为什么需要：JUnit通过这个注解识别测试方法

5.2.2 测试方法结构（Given-When-Then模式）

```
@Test  
void testLogin_Success() {  
    // Given - 准备测试数据  
    String username = "admin";  
    String password = "123456";  
    when(userMapper.findByUsername(username)).thenReturn(testUser);  
    when(passwordEncoder.matches(password,  
testUser.getPassword())).thenReturn(true);  
  
    // when - 执行被测方法  
    User result = userService.login(username, password);  
  
    // Then - 验证结果  
    assertNotNull(result);  
    assertEquals("admin", result.getUsername());  
}
```

说明:

- **Given**: 准备测试数据和Mock行为
- **When**: 调用被测方法
- **Then**: 验证结果是否符合预期

5.2.3 Mockito核心方法

```
when(userMapper.findByUsername(username)).thenReturn(testUser);
```

- 作用: 设置Mock对象的行为
- 含义: 当调用userMapper.findByUsername(username)时,返回testUser

```
verify(userMapper).findByUsername(username);
```

- 作用: 验证方法是否被调用
- 含义: 验证userMapper.findByUsername(username)是否被调用了一次

```
verify(passwordEncoder, never()).matches(anyString(), anyString());
```

- 作用: 验证方法从未被调用
- 含义: 验证passwordEncoder.matches()从未被调用

5.3 AuthController测试代码实现

```
package com.letao.sso.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.letao.entity.User;
import com.letao.sso.service.UserService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

/**
 * AuthController单元测试
 */
@WebMvcTest(AuthController.class)
class AuthControllerTest {

    @Autowired
```

```

private MockMvc mockMvc;

@Autowired
private ObjectMapper objectMapper;

@MockBean
private UserService userService;

@Test
void testLogin_Success() throws Exception {
    // Given
    User user = new User();
    user.setId(1L);
    user.setUsername("admin");

    user.setPassword("$2a$10$N.zmdr9k7uOCqb376NoUnuTJ8iAt6Z5EHsM8lE9lBosl7iAt6Z5EH")
;

    user.setStatus(1);

    when(userService.login("admin", "123456")).thenReturn(user);

    when(userService.findRolesByUserId(1L)).thenReturn(Arrays.asList("ROLE_ADMIN"));

    Map<String, String> loginParams = new HashMap<>();
    loginParams.put("username", "admin");
    loginParams.put("password", "123456");

    // When & Then
    mockMvc.perform(post("/auth/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(loginParams)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.code").value(200))
        .andExpect(jsonPath("$.message").value("登录成功"))
        .andExpect(jsonPath("$.data.user.username").value("admin"));
}

@Test
void testLogin_MissingUsername() throws Exception {
    // Given
    Map<String, String> loginParams = new HashMap<>();
    loginParams.put("password", "123456");

    // When & Then
    mockMvc.perform(post("/auth/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(loginParams)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.code").value(400))
        .andExpect(jsonPath("$.message").value("用户名或密码不能为空"));
}

@Test
void testLogin_InvalidCredentials() throws Exception {
    // Given
    when(userService.login("admin", "wrong")).thenReturn(null);

    Map<String, String> loginParams = new HashMap<>();

```

```
loginParams.put("username", "admin");
loginParams.put("password", "wrong");

// When & Then
mockMvc.perform(post("/auth/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(loginParams)))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.code").value(401))
    .andExpect(jsonPath("$.message").value("用户名或密码错误"));
}
```

5.4 Controller测试代码讲解

5.4.1 为什么需要Controller测试？

5.4.1.1 Controller层的独特职责

Controller层有以下独特的职责，需要专门测试：

AuthController的职责分析：

职责	说明	代码示例
接收HTTP请求	处理POST、GET等HTTP方法	<code>@PostMapping("/login")</code>
参数验证	验证请求参数是否有效	<code>if (username == null password == null)</code>
调用Service层	协调业务逻辑	<code>User user = userService.login(username, password)</code>
异常处理	捕获并处理异常，返回友好错误信息	<code>catch (Exception e)</code>
构建响应	将结果封装成统一的响应格式	<code>LetaoResult.ok("登录成功", result)</code>
跨域处理	处理CORS	<code>@CrossOrigin(origins = "*")</code>

5.4.1.2 Service测试不能替代Controller测试

核心问题：既然已经测试了Service层，为什么还要测试Controller层？

答案：因为Controller层和Service层测试的目标完全不同！

测试类型	测试目标	能否替代Controller测试	示例
Service测试	业务逻辑（用户验证、密码匹配）	✗ 不能	验证用户名密码是否正确
Controller测试	HTTP请求处理、参数验证、响应格式	☑ 必需	验证HTTP请求是否正确接收

具体举例：

```
// AuthController中的参数验证逻辑
if (username == null || password == null) {
    return LetaoResult.error(400, "用户名或密码不能为空");
}
```

这段代码：

- ☒ 在Service层不存在（Service假设参数已经验证）
- ☒ 只有Controller层有
- ☒ 必须通过Controller测试才能覆盖

5.4.1.3 Controller测试的价值

价值1：验证HTTP接口的正确性

验证项	说明	测试方法
请求路径	/api/sso/login是否正确	mockMvc.perform(post("/api/sso/login"))
请求方法	POST方法是否正确	MockMvcRequestBuilders.post()
请求参数	参数是否正确接收	content(objectMapper.writeValueAsString(loginParams))

价值2：验证参数验证逻辑

```
// 测试参数为null的场景
@Test
void testLogin_UsernameNull() {
    // username = null
    // 验证返回400错误
}
```

价值3：验证异常处理

```
// 测试Service抛出异常的场景
@Test
void testLogin_Exception() {
    // 模拟Service抛出异常
    when(userService.login(...)).thenThrow(new RuntimeException());
    // 验证Controller正确处理异常
}
```

价值4：验证响应格式

```
// 验证响应JSON结构
.andExpect(jsonPath("$.status").value(200))
.andExpect(jsonPath("$.msg").value("登录成功"))
.andExpect(jsonPath("$.data.token").exists())
```

5.4.1.4 Controller测试与Service测试的对比

对比维度	Controller测试	Service测试
测试对象	HTTP接口处理逻辑	业务逻辑
测试工具	MockMvc	直接调用方法
输入形式	HTTP请求 (JSON)	方法参数
输出形式	HTTP响应 (JSON)	返回值
关注点	参数验证、异常处理、响应格式	业务逻辑、数据验证
依赖Mock	@MockBean	@Mock
测试速度	较慢 (需要启动Web层)	较快 (纯单元测试)
测试环境	需要Web环境	不需要Web环境

5.4.2 Controller测试用例设计方法

5.4.2.1 白盒测试六步法 (Controller版)

Controller测试用例设计可以按照以下6个步骤进行：

步骤1：阅读和理解代码

↓

步骤2：识别代码结构特征

↓

步骤3：绘制控制流图

↓

步骤4：识别测试点

↓

步骤5：设计测试用例

↓

步骤6：优化和合并测试用例

5.4.2.2 步骤1：阅读和理解代码

被测代码：

```
@PostMapping("/login")
public LetaoResult<?> login(@RequestBody Map<String, String> loginParams) {
    try {
        // 1. 提取参数
        String username = loginParams.get("username");
        String password = loginParams.get("password");

        // 2. 参数验证
        if (username == null || password == null) {
            return LetaoResult.error(400, "用户名或密码不能为空");
        }

        // 3. 调用Service
        User user = userService.login(username, password);
        if (user == null) {
            return LetaoResult.error(401, "用户名或密码错误");
        }
    }
}
```



```
// 4. 查询角色
List<String> roles = userService.findRolesByUserId(user.getId());

// 5. 生成Token
String token = JwtUtils.generateToken(username, user.getId(),
roleArray);

// 6. 构建响应
Map<String, Object> result = new HashMap<>();
result.put("token", token);
result.put("user", user);
result.put("roles", roles);
return LetaoResult.ok("登录成功", result);

} catch (Exception e) {
    return LetaoResult.error(500, "登录失败: " + e.getMessage());
}
}
```

理解要点：

要素	说明
输入	HTTP POST请求，JSON格式的请求体
输出	LetaoResult格式的JSON响应
依赖	UserService（用户登录、角色查询）
关键逻辑	参数验证、Service调用、异常处理

5.4.2.3 步骤2：识别代码结构特征

结构特征识别清单：

结构特征	识别方法	AuthController.login方法
条件判断（if语句）	查找if、else if、三元运算符	2个if语句
方法调用	查找方法调用语句	3个Service方法调用
异常处理	查找try-catch、throw	1个try-catch
返回语句	查找return语句	4个return语句

详细分析：

条件判断：

- 第1个if: `if (username == null || password == null)` - 参数验证
- 第2个if: `if (user == null)` - 用户验证

方法调用：

- `loginParams.get("username")` - 获取用户名
- `loginParams.get("password")` - 获取密码
- `userService.login(username, password)` - 调用登录服务

- `userService.findRolesByUserId(user.getId())` - 查询角色
- `JwtUtils.generateToken(...)` - 生成Token

返回语句：

- `return LetaoResult.error(400, "用户名或密码不能为空");` - 参数错误
- `return LetaoResult.error(401, "用户名或密码错误");` - 登录失败
- `return LetaoResult.ok("登录成功", result);` - 登录成功
- `return LetaoResult.error(500, "登录失败: " + e.getMessage());` - 异常处理

5.4.2.4 步骤3：绘制控制流图

控制流图：



执行路径识别：

路径编号	执行路径	描述
路径1	1→2→3(否)→5→6(否)→8→9→10	登录成功
路径2	1→2→3(是)→4	参数为空
路径3	1→2→3(否)→5→6(是)→7	登录失败
路径4	1→2→3(否)→5→异常→11	Service抛出异常

5.4.2.5 步骤4：识别测试点

测试点识别方法：

测试点类型	识别方法	示例
参数验证	查找参数为null、空值的情况	username=null, password=null
Service调用	查找Service方法的返回值	userService.login()返回null或User对象
异常处理	查找可能抛出异常的代码	try-catch中的代码
响应格式	查找返回语句	验证响应的status、msg、data

测试点详细识别：

测试点1：参数验证

子测试点	具体内容	验证逻辑
测试点1.1	username为null	<code>username == null</code> 为真
测试点1.2	password为null	<code>password == null</code> 为真
测试点1.3	username和password都有效	两个条件都为假

测试点2：Service调用

子测试点	具体内容	验证逻辑
测试点2.1	userService.login()返回null	<code>user == null</code> 为真
测试点2.2	userService.login()返回User对象	<code>user == null</code> 为假

测试点3：异常处理

子测试点	具体内容	验证逻辑
测试点3.1	userService.login()抛出异常	进入catch分支
测试点3.2	userService.findRolesByUserId()抛出异常	进入catch分支
测试点3.3	JwtUtils.generateToken()抛出异常	进入catch分支

测试点4：响应格式

子测试点	具体内容	验证逻辑
测试点4.1	成功响应包含token、user、roles	验证data字段
测试点4.2	错误响应包含错误码和错误信息	验证status和msg字段

5.4.2.6 步骤5：设计测试用例

根据测试点，设计以下测试用例：

测试用例	测试场景	输入	预期输出	对应测试点	对应路径
testLogin_Success	登录成功	username="admin" password="123456"	status=200 msg="登录成功" 包含token、user、roles	测试点 2.2 测试点 4.1	路径1
testLogin_UsernameNull	用户名为空	username=null password="123456"	status=400 msg="用户名或密码不能为空"	测试点 1.1 测试点 4.2	路径2
testLogin_PasswordNull	密码为空	username="admin" password=null	status=400 msg="用户名或密码不能为空"	测试点 1.2 测试点 4.2	路径2
testLogin_InvalidCredentials	用户名或密码错误	username="admin" password="wrong"	status=401 msg="用户名或密码错误"	测试点 2.1 测试点 4.2	路径3
testLogin_Exception	Service抛出异常	username="admin" password="123456"	status=500 msg包含"登录失败"	测试点 3.1 测试点 4.2	路径4

5.4.2.7 步骤6：优化和合并测试用例

优化建议：

优化项	建议	原因
合并username和password为null的测试	✗ 不合并	它们是不同的参数验证逻辑
添加空字符串测试	☑ 可以添加	username=""、password=""
添加组合场景	☑ 可以添加	username和password都为null

最终测试用例列表：

- ☑ testLogin_Success - 登录成功
- ☑ testLogin_UsernameNull - 用户名为空
- ☑ testLogin_PasswordNull - 密码为空
- ☑ testLogin_InvalidCredentials - 用户名或密码错误
- ☑ testLogin_Exception - Service抛出异常

5.4.3 测试用例详细讲解

5.4.3.1 测试用例1：testLogin_Success

测试目的：验证登录成功场景

测试代码：

```
@Test
public void testLogin_Success() throws Exception {
    // 1. 准备测试数据
    User testUser = new User();
    testUser.setId("1");
    testUser.setUsername(TEST_USERNAME);
}
```

```

testUser.setPassword("encodedPassword");
testUser.setStatus(1);

List<String> roles = Arrays.asList("ADMIN", "USER");

Map<String, String> loginParams = new HashMap<>();
loginParams.put("username", TEST_USERNAME);
loginParams.put("password", TEST_PASSWORD);

// 2. 模拟依赖行为
when(userService.login(TEST_USERNAME, TEST_PASSWORD)).thenReturn(testUser);
when(userService.findRolesByUserId("1")).thenReturn(roles);

// 3. 执行测试
mockMvc.perform(post("/api/sso/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(new ObjectMapper().writeValueAsString(loginParams)))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.status").value(200))
    .andExpect(jsonPath("$.msg").value("登录成功"))
    .andExpect(jsonPath("$.data.token").exists())
    .andExpect(jsonPath("$.data.user").exists());

// 4. 验证依赖调用
verify(userService).login(TEST_USERNAME, TEST_PASSWORD);
verify(userService).findRolesByUserId("1");
}

```

详细讲解:

步骤1: 准备测试数据

```

User testUser = new User();
testUser.setId("1");
testUser.setUsername(TEST_USERNAME);
testUser.setPassword("encodedPassword");
testUser.setStatus(1);

```

说明:

- 创建一个模拟的User对象
- 设置必要的字段 (id、username、password、status)
- 这个User对象会被userService.login()返回

步骤2: 构建请求体

```

Map<String, String> loginParams = new HashMap<>();
loginParams.put("username", TEST_USERNAME);
loginParams.put("password", TEST_PASSWORD);

```

说明:

- 创建一个Map对象, 模拟HTTP请求的JSON body
- 设置username和password
- 这个Map会被序列化成JSON字符串

步骤3: 模拟依赖行为

```
when(userService.login(TEST_USERNAME, TEST_PASSWORD)).thenReturn(testUser);
when(userService.findRolesByUserId("1")).thenReturn(roles);
```

说明:

- 使用Mockito的when()方法模拟UserService的行为
- 当调用login()方法时, 返回testUser对象
- 当调用findRolesByUserId()方法时, 返回roles列表
- 这样可以避免调用真实的Service层

步骤4: 执行测试

```
mockMvc.perform(post("/api/sso/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(new ObjectMapper().writeValueAsString(loginParams)))
```

说明:

- 使用mockMvc.perform()发送HTTP请求
- post("/api/sso/login"): 发送POST请求到指定路径
- contentType(MediaType.APPLICATION_JSON): 设置请求内容类型为JSON
- content(...): 设置请求体, 将Map序列化成JSON字符串

步骤5: 验证响应

```
.andExpect(status().isOk())
.andExpect(jsonPath("$.status").value(200))
.andExpect(jsonPath("$.msg").value("登录成功"))
.andExpect(jsonPath("$.data.token").exists())
.andExpect(jsonPath("$.data.user").exists());
```

说明:

- status().isOk(): 验证HTTP状态码为200
- jsonPath("\$.status").value(200): 验证JSON响应中的status字段为200
- jsonPath("\$.msg").value("登录成功"): 验证JSON响应中的msg字段
- jsonPath("\$.data.token").exists(): 验证JSON响应中的data.token字段存在
- jsonPath("\$.data.user").exists(): 验证JSON响应中的data.user字段存在

步骤6: 验证依赖调用

```
verify(userService).login(TEST_USERNAME, TEST_PASSWORD);
verify(userService).findRolesByUserId("1");
```

说明:

- 使用Mockito的verify()方法验证依赖是否被调用
- verify(userService).login(...): 验证login()方法被调用了一次
- verify(userService).findRolesByUserId(...): 验证findRolesByUserId()方法被调用了一次

关键点总结:

关键点	说明
☒ 模拟UserService返回User对象	避免调用真实的Service层
☒ 模拟UserService返回角色列表	避免查询数据库
☒ 验证响应状态码为200	确保HTTP请求成功
☒ 验证响应包含token、user、roles	确保响应格式正确
☒ 验证UserService被正确调用	确保Controller正确调用Service

5.4.3.2 测试用例2: testLogin_UsernameNull

测试目的: 验证用户名为空时的参数验证

测试代码:

```
@Test
public void testLogin_UsernameNull() throws Exception {
    Map<String, String> loginParams = new HashMap<>();
    loginParams.put("username", null);
    loginParams.put("password", TEST_PASSWORD);

    mockMvc.perform(post("/api/sso/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(new ObjectMapper().writeValueAsString(loginParams)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.status").value(400))
        .andExpect(jsonPath("$.msg").value("用户名或密码不能为空"));

    verify(userService, never()).login(anyString(), anyString());
}
```

详细讲解:

步骤1: 构建请求体 (username为null)

```
Map<String, String> loginParams = new HashMap<>();
loginParams.put("username", null);
loginParams.put("password", TEST_PASSWORD);
```

说明:

- 创建一个Map对象
- 将username设置为null
- password设置为有效值

步骤2: 执行测试

```
mockMvc.perform(post("/api/sso/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(new ObjectMapper().writeValueAsString(loginParams)))
```

说明:

- 发送POST请求

- 请求体包含username=null

步骤3: 验证响应

```
.andExpect(status().isOk())
.andExpect(jsonPath("$.status").value(400))
.andExpect(jsonPath("$.msg").value("用户名或密码不能为空"));
```

说明:

- status().isOk(): 验证HTTP状态码为200 (注意: Controller返回200, 错误信息在响应体中)
- jsonPath("\$.status").value(400): 验证业务状态码为400
- jsonPath("\$.msg").value("用户名或密码不能为空"): 验证错误消息

步骤4: 验证依赖调用

```
verify(userService, never()).login(anyString(), anyString());
```

说明:

- verify(userService, never()): 验证UserService.login()从未被调用
- anyString(): 匹配任意字符串参数
- 这是因为参数验证失败, 不应该调用Service层

关键点总结:

关键点	说明
☒ username设置为null	模拟参数为空的场景
☒ 验证返回400错误码	确保参数验证生效
☒ 验证错误消息正确	确保返回友好的错误信息
☒ 验证UserService.login()未被调用	确保参数验证失败时不调用Service

为什么需要这个测试?

```
// Controller中的参数验证逻辑
if (username == null || password == null) {
    return LetaoResult.error(400, "用户名或密码不能为空");
}
```

这段代码:

- ☒ 只在Controller层存在
- ☒ Service层假设参数已经验证
- ☒ 必须通过Controller测试才能覆盖

5.4.3.3 测试用例3: testLogin_PasswordNull

测试目的: 验证密码为空时的参数验证

测试代码:

```
@Test
public void testLogin_PasswordNull() throws Exception {
```



```

Map<String, String> loginParams = new HashMap<>();
loginParams.put("username", TEST_USERNAME);
loginParams.put("password", null);

mockMvc.perform(post("/api/sso/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(new ObjectMapper().writeValueAsString(loginParams)))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.status").value(400))
    .andExpect(jsonPath("$.msg").value("用户名或密码不能为空"));

verify(userService, never()).login(anyString(), anyString());
}

```

详细讲解：

步骤1：构建请求体 (password为null)

```

Map<String, String> loginParams = new HashMap<>();
loginParams.put("username", TEST_USERNAME);
loginParams.put("password", null);

```

说明：

- 创建一个Map对象
- 将password设置为null
- username设置为有效值

步骤2：执行测试

```

mockMvc.perform(post("/api/sso/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(new ObjectMapper().writeValueAsString(loginParams)))

```

说明：

- 发送POST请求
- 请求体包含password=null

步骤3：验证响应

```

.andExpect(status().isOk())
.andExpect(jsonPath("$.status").value(400))
.andExpect(jsonPath("$.msg").value("用户名或密码不能为空"));

```

说明：

- 验证HTTP状态码为200
- 验证业务状态码为400
- 验证错误消息

步骤4：验证依赖调用

```

verify(userService, never()).login(anyString(), anyString());

```

说明：

- 验证UserService.login()从未被调用
- 确保参数验证失败时不调用Service

关键点总结：

关键点	说明
☒ password设置为null	模拟参数为空的场景
☒ 验证返回400错误码	确保参数验证生效
☒ 验证UserService.login()未被调用	确保参数验证失败时不调用Service

5.4.3.4 测试用例4：testLogin_InvalidCredentials

测试目的：验证用户名或密码错误场景

测试代码：

```
@Test
public void testLogin_InvalidCredentials() throws Exception {
    Map<String, String> loginParams = new HashMap<>();
    loginParams.put("username", TEST_USERNAME);
    loginParams.put("password", "wrongPassword");

    when(userService.login(TEST_USERNAME, "wrongPassword")).thenReturn(null);

    mockMvc.perform(post("/api/sso/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(new ObjectMapper().writeValueAsString(loginParams)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.status").value(401))
        .andExpect(jsonPath("$.msg").value("用户名或密码错误"));

    verify(userService).login(TEST_USERNAME, "wrongPassword");
    verify(userService, never()).findRolesById(anyString());
}
```

详细讲解：

步骤1：构建请求体（密码错误）

```
Map<String, String> loginParams = new HashMap<>();
loginParams.put("username", TEST_USERNAME);
loginParams.put("password", "wrongPassword");
```

说明：

- 创建一个Map对象
- username设置为有效值
- password设置为错误的值

步骤2：模拟依赖行为

```
when(userService.login(TEST_USERNAME, "wrongPassword")).thenReturn(null);
```

说明：

- 模拟UserService.login()返回null
- 表示用户名或密码错误

步骤3：执行测试

```
mockMvc.perform(post("/api/sso/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(new ObjectMapper().writeValueAsString(loginParams)))
```

说明：

- 发送POST请求
- 请求体包含错误的密码

步骤4：验证响应

```
.andExpect(status().isOk())
.andExpect(jsonPath("$.status").value(401))
.andExpect(jsonPath("$.msg").value("用户名或密码错误"));
```

说明：

- 验证HTTP状态码为200
- 验证业务状态码为401（未授权）
- 验证错误消息

步骤5：验证依赖调用

```
verify(userService).login(TEST_USERNAME, "wrongPassword");
verify(userService, never()).findRolesByUserId(anyString());
```

说明：

- verify(userService).login(...): 验证login()方法被调用了一次
- verify(userService, never()).findRolesByUserId(...): 验证findRolesByUserId()方法从未被调用
- 这是因为登录失败，不需要查询角色

关键点总结：

关键点	说明
☒ 模拟UserService.login()返回null	模拟登录失败场景
☒ 验证返回401错误码	确保返回正确的错误码
☒ 验证UserService.login()被调用	确保Controller正确调用Service
☒ 验证UserService.findRolesByUserId()未被调用	确保登录失败时不查询角色

5.4.3.5 测试用例5：testLogin_Exception

测试目的：验证Service抛出异常时的处理

测试代码：

```
@Test
public void testLogin_Exception() throws Exception {
```

```

Map<String, String> loginParams = new HashMap<>();
loginParams.put("username", TEST_USERNAME);
loginParams.put("password", TEST_PASSWORD);

when(userService.login(TEST_USERNAME, TEST_PASSWORD))
    .thenReturn(new RuntimeException("数据库连接异常"));

mockMvc.perform(post("/api/sso/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(new ObjectMapper().writeValueAsString(loginParams)))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.status").value(500))
    .andExpect(jsonPath("$.msg").exists());

verify(userService).login(TEST_USERNAME, TEST_PASSWORD);
verify(userService, never()).findRolesByUserId(anyString());
}

```

详细讲解:

步骤1: 构建请求体

```

Map<String, String> loginParams = new HashMap<>();
loginParams.put("username", TEST_USERNAME);
loginParams.put("password", TEST_PASSWORD);

```

说明:

- 创建一个Map对象
- username和password都设置为有效值

步骤2: 模拟依赖行为 (抛出异常)

```

when(userService.login(TEST_USERNAME, TEST_PASSWORD))
    .thenReturn(new RuntimeException("数据库连接异常"));

```

说明:

- 使用thenReturn()方法模拟抛出异常
- 模拟UserService.login()抛出RuntimeException
- 表示数据库连接异常

步骤3: 执行测试

```

mockMvc.perform(post("/api/sso/login")
    .contentType(MediaType.APPLICATION_JSON)
    .content(new ObjectMapper().writeValueAsString(loginParams)))

```

说明:

- 发送POST请求
- 请求体包含有效的用户名和密码

步骤4: 验证响应

```
.andExpect(status().isOk())
.andExpect(jsonPath("$.status").value(500))
.andExpect(jsonPath("$.msg").exists());
```

说明：

- status().isOk(): 验证HTTP状态码为200
- jsonPath("\$.status").value(500): 验证业务状态码为500（服务器错误）
- jsonPath("\$.msg").exists(): 验证错误消息存在（不验证具体内容，因为包含异常消息）

步骤5：验证依赖调用

```
verify(userService).login(TEST_USERNAME, TEST_PASSWORD);
verify(userService, never()).findRolesByUserId(anyString());
```

说明：

- verify(userService).login(...): 验证login()方法被调用了一次
- verify(userService, never()).findRolesByUserId(...): 验证findRolesByUserId()方法从未被调用
- 这是因为异常中断了流程，不会执行后续代码

关键点总结：

关键点	说明
☒ 模拟UserService.login()抛出异常	模拟Service层异常场景
☒ 验证返回500错误码	确保异常被正确处理
☒ 验证异常被正确捕获和处理	确保返回友好的错误信息
☒ 验证UserService.findRolesByUserId()未被调用	确保异常中断流程

为什么需要这个测试？

```
// Controller中的异常处理逻辑
try {
    User user = userService.login(username, password);
    // ...
} catch (Exception e) {
    return LetaoResult.error(500, "登录失败: " + e.getMessage());
}
```

这段代码：

- ☒ 只在Controller层存在
 - ☒ 需要验证异常是否被正确捕获
 - ☒ 需要验证是否返回友好的错误信息
 - ☒ 必须通过Controller测试才能覆盖
-

5.4.4 Controller测试的核心注解和工具

5.4.4.1 @WebMvcTest注解

```
@WebMvcTest(AuthController.class)
```

作用：

- 只加载Controller层，不加载整个Spring上下文
- 自动配置MockMvc
- 只扫描指定的Controller类

为什么需要：

- ☒ 提高测试速度（不加载整个应用）
- ☒ 只测试Controller层（隔离测试）
- ☒ 自动配置测试环境（简化配置）

对比：

注解	加载范围	测试速度	适用场景
@WebMvcTest	只加载Controller层	快	Controller层测试
@SpringBootTest	加载整个Spring上下文	慢	集成测试

5.4.4.2 MockMvc

```
@Autowired
private MockMvc mockMvc;
```

作用：

- 提供模拟HTTP请求的工具
- 可以发送GET、POST、PUT、DELETE等请求
- 可以验证HTTP响应

为什么需要：

- ☒ 测试Controller需要模拟HTTP请求
- ☒ 避免启动真实的HTTP服务器
- ☒ 可以精确控制请求和验证响应

常用方法：

方法	说明	示例
perform()	执行HTTP请求	mockMvc.perform(post("/api/sso/login"))
andExpect()	验证响应	.andExpect(status().isOk())
andDo()	打印请求和响应	.andDo(print())

5.4.4.3 @MockBean

```
@MockBean
private UserService userService;
```

作用：

- 创建UserService的Mock对象
- 替换Spring容器中的Bean
- 可以模拟依赖对象的行为

为什么需要：

- ☒ 隔离测试（避免调用真实的Service层）
- ☒ 控制依赖行为（可以设置返回值）
- ☒ 提高测试速度（不需要访问数据库）

对比：

注解	作用	使用场景
@MockBean	创建Mock对象并替换Spring容器中的Bean	Spring环境中的测试
@Mock	创建Mock对象但不替换Spring容器中的Bean	纯单元测试

5.4.5 MockMvc使用方法详解

5.4.5.1 发送HTTP请求

基本语法：

```
mockMvc.perform(
    MockMvcRequestBuilders.post("/api/sso/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(loginParams))
)
```

详细说明：

方法	说明	示例
MockMvcRequestBuilders.post()	发送POST请求	post("/api/sso/login")
MockMvcRequestBuilders.get()	发送GET请求	get("/api/sso/test")
MockMvcRequestBuilders.put()	发送PUT请求	put("/api/sso/update")
MockMvcRequestBuilders.delete()	发送DELETE请求	delete("/api/sso/delete")

设置请求头：

方法	说明	示例
contentType()	设置Content-Type	.contentType(MediaType.APPLICATION_JSON)
accept()	设置Accept	.accept(MediaType.APPLICATION_JSON)
header()	设置请求头	.header("Authorization", "Bearer token")

设置请求体：

方法	说明	示例
content()	设置请求体	.content(objectMapper.writeValueAsString(loginParams))
param()	设置请求参数	.param("username", "admin")

5.4.5.2 验证HTTP响应

基本语法：

```
.andExpect(status().isOk())
.andExpect(jsonPath("$.status").value(200))
.andExpect(jsonPath("$.msg").value("登录成功"))
```

验证HTTP状态码：

方法	说明	示例
status().isOk()	验证状态码为200	.andExpect(status().isOk())
status().isCreated()	验证状态码为201	.andExpect(status().isCreated())
status().isBadRequest()	验证状态码为400	.andExpect(status().isBadRequest())
status().isUnauthorized()	验证状态码为401	.andExpect(status().isUnauthorized())
status().isNotFound()	验证状态码为404	.andExpect(status().isNotFound())
status().isInternalServerError()	验证状态码为500	.andExpect(status().isInternalServerError())

验证JSON响应：

方法	说明	示例
jsonPath("\$.status").value(200)	验证status字段	.andExpect(jsonPath("\$.status").value(200))
jsonPath("\$.msg").value("登录成功")	验证msg字段	.andExpect(jsonPath("\$.msg").value("登录成功"))
jsonPath("\$.data.token").exists()	验证字段存在	.andExpect(jsonPath("\$.data.token").exists())
jsonPath("\$.data.user.username").value("admin")	验证嵌套字段	.andExpect(jsonPath("\$.data.user.username").value("admin"))

验证响应头：

方法	说明	示例
header().string("Content-Type", "application/json")	验证响应头	.andExpect(header().string("Content-Type", "application/json"))

5.4.5.3 打印请求和响应

```
.andDo(print())
```

作用：

- 打印HTTP请求的详细信息
- 打印HTTP响应的详细信息
- 用于调试测试

输出示例：

```
MockHttpServletRequest:
    HTTP Method = POST
    Request URI = /api/sso/login
    Parameters = {}
    Headers = [Content-Type=application/json]
    Body = {"username":"admin","password":"123456"}
    Session Attrs = {}

MockHttpServletResponse:
    Status = 200
    Error message = null
    Headers = [Content-Type=application/json;charset=UTF-8]
    Content type = application/json;charset=UTF-8
    Body = {"status":200,"msg":"登录成功","data":{"token":"...","user":
{...},"roles":[...]}}
    Forwarded URL = null
    Redirected URL = null
    Cookies = []
```

5.4.6 Controller测试最佳实践

5.4.6.1 测试命名规范

```
// 好的命名
testLogin_Success()
testLogin_UsernameNull()
testLogin_PasswordNull()
testLogin_InvalidCredentials()
testLogin_Exception()

// 不好的命名
test1()
testLogin()
testMethod()
```

规范：test + 被测方法名 + 测试场景

5.4.6.2 测试独立性

```
// 好的实践
@Test
void testLogin_Success() {
    Map<String, String> loginParams = new HashMap<>();
    loginParams.put("username", TEST_USERNAME);
    loginParams.put("password", TEST_PASSWORD);
    // ...
}

// 不好的实践
```

```
private Map<String, String> loginParams = new HashMap<>();

@Test
void testLogin_Success() {
    loginParams.put("username", TEST_USERNAME);
    loginParams.put("password", TEST_PASSWORD);
    // ...
}
```

规范：每个测试方法都独立准备数据，避免共享变量

5.4.6.3 测试可读性

```
// 好的实践：使用Given-When-Then模式
@Test
void testLogin_Success() {
    // Given
    Map<String, String> loginParams = new HashMap<>();
    loginParams.put("username", TEST_USERNAME);
    loginParams.put("password", TEST_PASSWORD);
    when(userService.login(TEST_USERNAME, TEST_PASSWORD)).thenReturn(testUser);

    // when
    mockMvc.perform(post("/api/sso/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(new ObjectMapper().writeValueAsString(loginParams)))

    // Then
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.status").value(200));
}

// 不好的实践：没有注释，逻辑混乱
@Test
void testLogin_Success() {
    Map<String, String> loginParams = new HashMap<>();
    loginParams.put("username", TEST_USERNAME);
    loginParams.put("password", TEST_PASSWORD);
    when(userService.login(TEST_USERNAME, TEST_PASSWORD)).thenReturn(testUser);
    mockMvc.perform(post("/api/sso/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(new ObjectMapper().writeValueAsString(loginParams)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.status").value(200));
}
```

规范：使用Given-When-Then模式，提高测试可读性

5.4.6.4 测试覆盖率

目标覆盖率：

指标	目标值	说明
行覆盖率	> 80%	每行代码都被测试到
分支覆盖率	> 70%	每个if分支都被测试到
方法覆盖率	> 80%	每个方法都被测试到

如何提高覆盖率：

- 1. ☒ 覆盖所有执行路径（成功路径、失败路径、异常路径）
- 2. ☒ 覆盖所有条件判断（if语句的真假分支）
- 3. ☒ 覆盖所有边界值（null、空字符串、有效值）
- 4. ☒ 覆盖所有异常情况（Service抛出异常）

5.4.7 常见问题解答

Q1: Controller测试需要启动数据库吗？

A：不需要。Controller测试使用@MockBean模拟Service层，不需要访问数据库。

Q2: Controller测试和集成测试有什么区别？

A：

对比维度	Controller测试	集成测试
测试范围	只测试Controller层	测试多个层的集成
依赖Mock	使用@MockBean	使用真实Bean
数据库	不需要	需要
测试速度	快	慢

Q3: 什么时候使用@WebMvcTest，什么时候使用@SpringBootTest？

A：

注解	使用场景
@WebMvcTest	只测试Controller层，需要快速测试
@SpringBootTest	测试多个层的集成，需要完整的环境

Q4: 如何测试文件上传？

A：

```
mockMvc.perform(multipart("/api/upload")
    .file("file", file.getBytes())
    .param("description", "test"))
    .andExpect(status().isOk());
```

Q5: 如何测试需要认证的接口？

A：

```
mockMvc.perform(get("/api/user/profile")
    .header("Authorization", "Bearer " + token))
    .andExpect(status().isOk());
```

5.4.8 总结

5.4.8.1 为什么需要Controller测试?

1. ☒ **Controller层有独特的职责**: 参数验证、异常处理、响应格式化
2. ☒ **Service测试无法覆盖**: HTTP请求处理、参数验证逻辑
3. ☒ **验证接口正确性**: 确保API对外暴露的行为符合预期
4. ☒ **保护集成点**: Controller是前后端交互的入口, 需要保证稳定性

5.4.8.2 Controller测试用例如何设计?

1. ☒ **应用白盒测试六步法**: 理解代码→识别结构→绘制流图→识别测试点→设计用例→优化用例
2. ☒ **关注Controller特有逻辑**: 参数验证、异常处理、响应格式
3. ☒ **覆盖所有执行路径**: 成功路径、失败路径、异常路径
4. ☒ **验证依赖调用**: 确保Service层被正确调用

5.4.8.3 Controller测试的价值

- ☒ **快速反馈**: 在开发阶段发现接口问题
- ☒ **文档作用**: 测试用例本身就是接口文档
- ☒ **重构保护**: 修改代码时, 测试可以保证接口行为不变
- ☒ **集成验证**: 验证Controller与Service的集成是否正确

6. 测试执行与验证

6.1 执行测试

6.1.1 使用IDE执行

IntelliJ IDEA:

1. 右键点击测试类或测试方法
2. 选择"Run '测试类名'"或"Run '测试方法名'"
3. 查看测试结果

6.1.2 使用Maven执行

```
# 执行所有测试
mvn test

# 执行指定测试类
mvn test -Dtest=UserServiceImplTest

# 执行指定测试方法
mvn test -Dtest=UserServiceImplTest#testLogin_Success
```

6.2 查看测试结果

6.2.1 测试通过

```
[INFO] -----  
[INFO]  T E S T S  
[INFO] -----  
[INFO] Running com.letao.sso.service.impl.UserServiceImplTest  
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  
[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] -----
```

说明:

- Tests run: 5 - 运行了5个测试方法
- Failures: 0 - 没有失败的测试
- Errors: 0 - 没有错误的测试
- Skipped: 0 - 没有跳过的测试

6.2.2 测试失败

```
[INFO] -----  
[INFO]  T E S T S  
[INFO] -----  
[INFO] Running com.letao.sso.service.impl.UserServiceImplTest  
[ERROR] Tests run: 5, Failures: 1, Errors: 0, Skipped: 0  
[INFO] -----  
[ERROR] testLogin_Success Time elapsed: 0.05 sec <<< FAILURE!  
org.opentest4j.AssertionFailedError:  
Expected :admin  
Actual   :null  
        at  
com.letao.sso.service.impl.UserServiceImplTest.testLogin_Success(UserServiceImpl  
Test.java:XX)  
[INFO] -----  
[INFO] BUILD FAILURE  
[INFO] -----
```

说明:

- Failures: 1 - 有1个测试失败
- Expected :admin - 期望返回"admin"
- Actual :null - 实际返回null
- 需要检查代码逻辑或测试用例

6.3 测试覆盖率

6.3.1 添加JaCoCo插件

在 `pom.xml` 中添加JaCoCo插件:

```
<plugin>  
  <groupId>org.jacoco</groupId>  
  <artifactId>jacoco-maven-plugin</artifactId>
```

```
<version>0.8.7</version>
<executions>
  <execution>
    <goals>
      <goal>prepare-agent</goal>
    </goals>
  </execution>
  <execution>
    <id>report</id>
    <phase>test</phase>
    <goals>
      <goal>report</goal>
    </goals>
  </execution>
</executions>
</plugin>
```

6.3.2 生成覆盖率报告

```
# 执行测试并生成覆盖率报告
mvn clean test
```

报告生成在: `target/site/jacoco/index.html`

6.3.3 查看覆盖率报告

打开 `target/site/jacoco/index.html`, 可以看到:

指标	说明	目标值
Instructions	字节码覆盖率	> 80%
Branches	分支覆盖率	> 70%
Lines	行覆盖率	> 80%
Methods	方法覆盖率	> 80%
Classes	类覆盖率	> 80%

6.4 测试最佳实践

6.4.1 测试命名规范

```
// 好的命名
testLogin_Success()
testLogin_UserNotFound()
testLogin_WrongPassword()

// 不好的命名
test1()
testLogin()
testMethod()
```

规范: `test + 被测方法名 + 测试场景`

6.4.2 测试独立性

```
// 好的实践
@Test
void testLogin_Success() {
    // 每个测试方法都独立准备数据
    String username = "admin";
    String password = "123456";
    // ...
}

// 不好的实践
private String username = "admin"; // 共享变量
private String password = "123456";

@Test
void testLogin_Success() {
    // 依赖共享变量,可能导致测试之间互相影响
    // ...
}
```

6.4.3 测试可读性

```
// 好的实践: 使用Given-when-Then模式
@Test
void testLogin_Success() {
    // Given
    String username = "admin";
    String password = "123456";
    when(userMapper.findByUsername(username)).thenReturn(testUser);

    // when
    User result = userService.login(username, password);

    // Then
    assertNotNull(result);
}

// 不好的实践: 没有注释,逻辑混乱
@Test
void testLogin_Success() {
    String username = "admin";
    String password = "123456";
    when(userMapper.findByUsername(username)).thenReturn(testUser);
    User result = userService.login(username, password);
    assertNotNull(result);
}
```

附录A: 常见问题解答

Q1：什么时候使用Mock,什么时候使用真实对象？

A:

- **使用Mock:**
 - 依赖外部资源（数据库、网络、文件系统）
 - 依赖不稳定或不可控的对象
 - 需要控制依赖对象的行为
 - 需要隔离测试
- **使用真实对象:**
 - 简单的POJO对象
 - 不依赖外部资源的工具类
 - 需要测试对象之间的真实交互

Q2：如何选择测试覆盖标准？

A:

- **语句覆盖:** 快速验证,适合简单代码
- **判定覆盖:** 一般项目,适合大多数场景
- **条件覆盖:** 复杂逻辑,适合包含复合条件的代码
- **基本路径覆盖:** 核心业务,适合重要功能

Q3：测试用例越多越好吗？

A: 不是。测试用例应该:

- ☒ 覆盖所有关键逻辑
- ☒ 覆盖边界条件
- ☒ 覆盖异常情况
- ☒ 避免重复测试
- ☒ 避免过度测试

Q4：如何处理测试中的异常？

A:

```
@Test
void testLogin_ThrowsException() {
    // Given
    when(userMapper.findByUsername(anyString()))
        .thenReturn(new RuntimeException("Database error"));

    // When & Then
    assertThrows(RuntimeException.class, () -> {
        userService.login("admin", "123456");
    });
}
```

附录B：参考资料

B.1 官方文档

- [JUnit 5 User Guide](#)
- [Mockito Documentation](#)
- [Spring Boot Test Documentation](#)

B.2 推荐书籍

- 《Effective Unit Testing》- Lasse Koskela
- 《Test-Driven Development》- Kent Beck
- 《Growing Object-Oriented Software, Guided by Tests》- Steve Freeman

B.3 在线资源

- [Baelding - JUnit 5 Guide](#)
- [Baelding - Mockito Guide](#)
- [JUnit 5 GitHub](#)

总结

本教程从零开始,系统地讲解了白盒测试的概念、方法和实践。通过Letao SSO登录功能的实战演练,您应该已经掌握了:

1. ☒ 白盒测试的基本概念和覆盖标准
2. ☒ 测试环境的搭建和依赖配置
3. ☒ 白盒测试用例设计的六步法
4. ☒ 使用JUnit和Mockito编写测试代码
5. ☒ 测试执行和覆盖率分析

下一步建议:

- 在实际项目中应用白盒测试方法
- 学习更多测试技巧和最佳实践
- 探索集成测试和端到端测试
- 持续改进测试代码质量

祝您测试愉快! 🍀